



GM BA3 2025

Programmation pour Ingénieur

Billard 2025



Date de Rendu : 28 Décembre 2025

Diebold Jules : 393356
Delamare Romain : 373041
Bancken Thomas : 393134

1 Introduction

Dans le cadre de notre cours de **Programmation pour ingénieur**, nous réalisons ici un projet qui vise à l'analyse d'une séquence vidéo de billard afin de déterminer si un joueur marque un point ou non. Pour atteindre cet objectif, le projet va être décomposé en 3 parties distinctes chacun utilisant un environnement de cours différent (Code C, Matlab et Labview). Ce document a pour but d'expliquer notre démarche et la mise en œuvre de celle-ci.

Informations Complémentaires

- **OS + version** : MacOS Sonoma 14.5
- **Plateforme de test** : MacBook Air M1 (Apple Silicon)
- **Compilateur C + version** : VS Code 1.87.0
- **Matlab + version** : R2025b Apple silicon
- **LabVIEW + version** : 2025Q3

2 Pix2Pos.c

Le programme **Pix2Pos.c** a été développé en langage C sous l'environnement VS Code et compilé sur la plateforme MacOS. Son rôle est d'analyser les données d'image pour déterminer les coordonnées X, Y et le score de chacune des balles. Ce programme est appelé par LabVIEW et commence par lire le fichier binaire **Pixmap.bin**, qui contient l'intégralité des données de pixels de l'image. Le programme parcourt le tableau de pixels en mémoire pour analyser les composantes de couleur et trouver le carré de pixels qui présente le meilleur score. Il crée ensuite le fichier texte **Pos.txt**, qui contient les coordonnées X et Y de la position des boules, ainsi que le score associé. Enfin, Pix2Pos.c renvoie différents types d'**erreurs** et de **warnings** qui vont être gérés par LabVIEW.

Explication des fonctions utilisées

1. **void conversion** (unsigned int Pix, struct pixel *RGB)

Extrait les composantes Rouge, Vert et Bleu par **shifting** et **masking** de la valeur entière Pix qui encode un pixel sous la forme 0x00RRGGBB. Les composantes sont ensuite stockées dans la structure pointée par RGB.

2. **int verification couleur** (unsigned int Pix, struct range couleur)

Utilise les composantes RGB données par conversion puis compare chaque composante aux bornes min/max définies dans la structure "couleur".

3. **void score** (unsigned int Pix, struct range JauneR, struct range BlancR, struct range RougeR, int x, int y, unsigned int largeur, int *ScoreJaune, int *ScoreBlanc, int *ScoreRouge, int BallDiameter)

Analyse un carré de taille BallDiameter × BallDiameter dont le coin supérieur gauche est situé en (x, y). Pour chaque pixel de cette zone, la fonction vérifie son appartenance à la plage RGB de chaque couleur et incrémente les compteurs associés.

4. `void scoremax` (unsigned int *Pix, unsigned int largeur, struct range JauneR, struct range BlancR, struct range RougeR, int Lmin, int Lmax, int Cmin, int Cmax, int BallDiameter, struct donnees *JauneSP, struct donnees *BlancSP, struct donnees *RougeSP)

Balaye la zone définie par les limites Lmin–Lmax et Cmin–Cmax. Pour chaque position possible du carré de taille BallDiameter, la fonction calcule les scores de couleurs via la fonction "score". Les meilleurs scores pour chaque couleur sont enregistrés avec la position associée dans les structures JauneSP, RougeSP et BlancSP. Si aucune zone ne dépasse le score minimal Ballmin-Score, la boule est considérée comme "non détectée" et ses coordonnées sont fixées à (-1, -1).

5. `int main`(int argc, char *argv[])

La fonction main de l'exécutable Pix2Pos est chargée de l'initialisation et de la gestion du flux de données. Elle reçoit les 29 paramètres nécessaires à l'analyse transmis par LabVIEW. Elle commence par lire le fichier binaire Pixmap.bin pour ensuite allouer dynamiquement la mémoire pour les pixels. main appelle ensuite la logique d'analyse (qui utilise les fonctions précédentes) pour calculer les coordonnées X et Y et le score de chaque boule. Finalement, elle crée et écrit les résultats dans le fichier Pos.txt, tout en renvoyant les codes d'erreurs et de warnings.

Erreurs

- **return 1: Nombre d'arguments incorrect dans la ligne de commande**
- **return 2: Valeur incorrecte pour BallDiameter**
- **return 3: Fichier Pixmap.bin non ouvrable**
- **return 4: Largeur manquante**
- **return 5: Hauteur manquante**
- **return 6: Dimensions hors bornes**
- **return 7: Attribution de mémoire impossible**
- **return 8: Pas assez de pixels dans Pixmap.bin**
- **return 9: Fichier Pos.txt non ouvrable**
- **return 10: Ecriture dans Pos.txt impossible**
- **return 11: Fermeture de Pos.txt impossible**

Warnings

- Trop de pixels détectés
- Une boule n'est pas détectée

3 AnalyseTx.m

Le script MATLAB est utilisé pour créer le bilan de la partie au format PDF et déterminer combien de boules et de bandes ont été touchées durant la partie. La feuille de score montre quelle boule a été touchée en premier et si la partie a été gagnée ou non. L'utilisateur peut choisir de modifier plusieurs éléments de la feuille de score depuis LabVIEW.

Explication des fonctions utilisées

1. **function** [Xmin,Xmax,Ymin,Ymax] = GetFrame(Xr,Yr,Xy,Yy,Xw,Yw)

La fonction utilise les coordonnées de chacune des trois boules pour trouver les coordonnées maximales et minimales de leurs positions. Cela permet de déduire et de fixer les bords du billard.

2. **function** PathLength = GetBallPathLength(X,Y)

La fonction calcule la longueur totale du chemin parcouru par une boule. Cette information est cruciale, car elle permet de déterminer si une boule a effectivement bougé, ce qui est une information nécessaire pour valider la condition de victoire.

3. **function** [FirstMoveIdx, MoveDist] = GetFirstMoveIdx(X,Y, MoveDistPx)

La fonction calcule la distance parcourue pour chaque segment de la trajectoire de la boule, générant le vecteur MoveDist. Elle compare ces distances à la distance minimale MoveDistPx pour déterminer le FirstMoveIdx. Elle est essentielle pour identifier la première boule à bouger de la partie.

4. **function** [IdxTouch, offset] = GetTouchIdx(X,Y,Xmin, Xmax, Ymin, Ymax, BallBorderDist)

Son rôle est de déterminer les indices où la boule entre en contact avec l'une des bandes du billard. Elle vérifie si la position de la boule est à une distance inférieure à BallBorderDist de l'une des limites. Elle retourne le vecteur IdxTouch contenant les indices des rebonds

5. **function** [FirstBall,SecondBall,LastBall, NbBallsMoved] = GetBallMoveOrder(Xr, Yr, Xy, Yy, Xw, Yw, MoveDistPx)

Cette fonction permet de déterminer le nombre de boules qui ont bougé et l'ordre dans lequel elles ont bougé. Si deux boules bougent en même temps, on dit que la première boule qui bouge est celle qui a la distance initiale la plus grande.

6. **function** [X,Y]=InterpolateNan(X,Y)

Quand une boule n'est pas détectée, LabVIEW attribue NAN aux coordonnées X et Y. Cette fonction permet de détecter les NAN et des les remplacer par la méthode interp1.

7. **function** [X,Y]=RemoveOutlier(X,Y)

Elle utilise la méthode de la médiane mobile en MATLAB pour déterminer quand les coordonnées X et Y sont aberrantes en même temps. La fonction remplace alors les coordonnées aberrantes par la dernière bonne coordonnée connue de la boule.

4 Billard.vi

Le VI principal **Billard.vi** assure la coordination entre le code Matlab et le code C. L'utilisateur commence par renseigner le chemin du dossier contenant l'ensemble des images de la partie de billard. Une vérification est effectuée afin de s'assurer que le dossier n'est pas vide et qu'il contient uniquement des images au format PNG. En cas de problème, le programme s'arrête directement et renvoie un code et un message d'erreur à l'utilisateur.

Une fois ces vérifications réalisées, les limites de la table de billard sont déterminées à l'aide de la sous-VI **Limites table**. Ces limites sont nécessaires pour délimiter la zone d'analyse de la partie.

Le programme exécute ensuite une boucle itérative sur l'ensemble des images, permettant d'extraire de chaque image, les coordonnées des différentes boules présentes sur la table. Dans cette boucle, on vérifie aussi qu'aucune erreur ne ressort du code C. Dans ce cas, le programme s'arrête et affiche un message d'erreur sur l'interface utilisateur.

Les données de coordonnées sont ensuite transmises à MATLAB. Plusieurs coordonnées peuvent être erronées, c'est pourquoi on remplace les 'NAN' par la prochaine coordonnée valide et les valeurs aberrantes par la valeur précédente. A partir de ces données corrigées, MATLAB génère un fichier PDF retraçant l'intégralité de la partie de billard, incluant les trajectoires des boules ainsi qu'un résumé de la partie. Ceci est renvoyé à LabVIEW et affiché à l'utilisateur.

Erreurs

- **Error 12 : Can't open folder or folder not found**
- **Error 13 : Wrong folder name (TX)**
- **Error 14 : PNG folder empty or not found**
- **Error 1-11 : Problème dans le code C**